

UCS503 – Software Engineering Project

Academic Year 2026–2027

Nursify AI

Intelligent Audio-Based Clinical Handoff System

Voice-Driven Handoff Capture, Speech-to-Text and AI-Assisted Clinical Information Retrieval

Mid-Semester Evaluation Report — Revised Edition

(Updated with Collaboration & Class Diagrams, SRS, User Stories and Current Prototype Screens)

Submitted by:

1024031035 Harshav Dhawan

1024031033 Anshuman

1024031101 Harnav Singh

B.Tech Third Year, Computer Engineering

Submitted to:

Dr. Nisha

Department of Computer Science and Engineering

Thapar Institute of Engineering and Technology, Patiala

2026

Contents

Chapter 1 Introduction.....	3
1.1 Background and Context.....	3
1.2 Problem Statement.....	3
1.3 Project Objectives.....	3
Chapter 2 Methodology and Design.....	5
2.1 System Architecture.....	5
2.1.1 Use Case Diagram.....	5
2.1.2 Use Case Templates.....	5
2.1.3 High-Level Design.....	8
2.1.4 Sequence Diagram.....	9
2.1.5 Collaboration Diagram.....	9
2.1.6 Class Diagram.....	10
2.1.7 Entity-Relationship Diagram.....	11
2.1.8 Technical Stack.....	12
2.2 Data Flow Design.....	12
2.2.1 Handoff Processing Pipeline.....	13
2.3 Software Requirement Specification (IEEE Format).....	15
2.3.1 Introduction.....	15
2.3.2 Overall Description.....	15
2.3.3 Specific Requirements.....	16
2.4 User Stories and Story Cards.....	18
Chapter 3 Implementation and Evaluation.....	20
3.1 Implementation Status.....	20
3.1.1 FastAPI Backend and CORS.....	20
3.1.2 Speech-to-Text Implementation.....	20
3.1.3 RAG and Vector Search Design.....	21
3.1.4 Prototype User Interface (Current Build).....	21
3.2 Testing Methodology.....	22
3.3 Current Results.....	23
Chapter 4 Conclusion.....	24
4.1 Summary of Work.....	24
4.2 Key Findings.....	24
4.3 Future Work and Recommendations.....	24
4.4 Final Remarks.....	25
Bibliography.....	26

Chapter 1

Introduction

1.1 Background and Context

Clinical handoffs are a critical part of nursing workflow because responsibility for a patient is transferred from one caregiver or shift to another. In practice, handoff information may be communicated verbally while also being documented manually. This creates a repetitive documentation burden and can make it difficult for the incoming nurse to quickly reconstruct what changed, what remains pending, and which observations require attention.

Nursify AI is designed as a voice-driven clinical handoff system. The central idea is to allow a nurse to speak a handoff naturally, capture the audio, convert the speech into text, and progressively transform the information into structured and searchable clinical records. The proposed system combines a cross-platform application, a FastAPI backend, speech-to-text processing, clinical NLP/LLM components, PostgreSQL storage, and retrieval-augmented question answering.

1.2 Problem Statement

Manual clinical handoff documentation requires nurses to divide attention between patient care and data entry. A purely verbal handoff can also be difficult to search or review later, while unstructured notes can omit important fields such as medications, allergies, vitals, tests, or pending actions.

Nursify AI addresses this problem by introducing an audio-first workflow in which the spoken handoff becomes the starting point for structured documentation. The current delivered prototype demonstrates the end-to-end speech-to-text stage together with a working application shell (dashboard, task manager, and handoff screens) that the remaining AI stages will plug into. The broader architecture provides a staged path toward clinical entity extraction, SBAR structuring, timeline generation, missing-information detection, risk decision support, and RAG-based patient-specific question answering.

1.3 Project Objectives

The core objectives of Nursify AI are:

- Voice-based handoff capture: allow a nurse to record a bedside handoff without manually typing the entire interaction.
- Speech-to-text transcription: convert recorded clinical speech into a readable transcript using a local Whisper-family model.
- Clinical structuring: extract useful medical entities such as vitals, medications, diagnoses, allergies, and tests from the transcript in later stages.
- Automated handoff organization: generate SBAR-style handoffs, concise summaries, and timestamped timeline events.
- Decision support: experiment with completeness checking and deterioration-risk scoring while keeping the nurse responsible for verification.
- Patient-specific AI assistance: use retrieval-augmented generation so questions can be answered from relevant patient records rather than unsupported model knowledge.

- Usable task management: surface AI-prioritised, time-sensitive nursing tasks (medication administration, vitals checks, assessments) alongside the handoff workflow so the handoff and the shift's to-do list stay in sync.

Safety boundary: Nursify AI is intended as a documentation and decision-support system. Its summaries, retrieved answers, and experimental risk outputs are not intended to replace clinical judgment or provide autonomous diagnosis.

Chapter 2

Methodology and Design

2.1 System Architecture

2.1.1 Use Case Diagram

Before describing the layered architecture, the use case diagram summarises the system from the nurse's point of view. It groups the nurse-facing functionality into authentication and patient browsing, voice handoff capture and processing (record, upload, and the included sub-processes of SBAR generation, summary generation, timeline generation, and missing-information checking), and review functionality such as viewing the AI summary, patient timeline, risk indicator, editing a handoff manually, asking the AI assistant a question, and viewing handoff history.

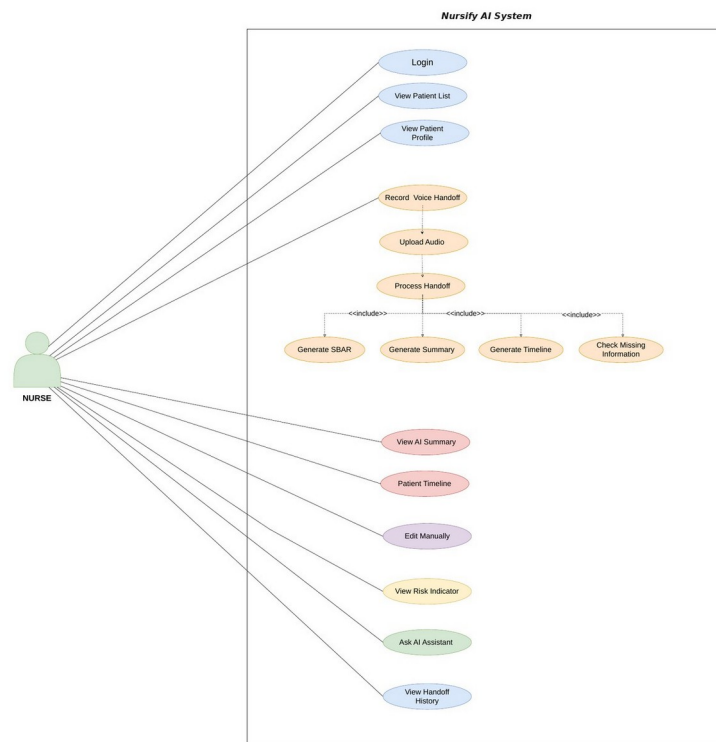


Figure 2.1: Nursify AI System – Use Case Diagram

2.1.2 Use Case Templates

Detailed use-case templates for the six most significant nurse-facing use cases are provided below, following the standard actor / pre-condition / main-flow / alternate-flow / post-condition format used for the mid-semester analysis.

Use Case ID / Name	UC-01 / Record Voice Handoff
Actor(s)	Nurse
Description	A nurse selects a patient and records a spoken bedside handoff, which is uploaded to the backend for processing.
Pre-conditions	The nurse is authenticated and a patient has been selected from the patient list.
Main Flow	1. Nurse selects a patient. 2. Nurse taps "Record Handoff". 3. Mobile/desktop client records audio. 4. Client uploads the audio file with the patient ID to the backend (UC-02, include).
Alternate / Exception Flow	If the upload fails (e.g., network loss), the client retains the local recording and retries or lets the nurse resend later.
Post-conditions	Audio is stored and a handoff-processing job is triggered.
Includes / Extends	Includes UC-02.

Use Case ID / Name	UC-02 / Process Handoff
Actor(s)	System (internal)
Description	Converts the uploaded audio into a transcript and structures it into SBAR fields, a summary, and timeline events.
Pre-conditions	Audio has been received from UC-01.
Main Flow	1. Backend sends audio to the Speech-to-Text service. 2. Transcript is returned. 3. Transcript is passed to the NLP & Handoff Processor. 4. Structured data (SBAR, entities, timeline events) is generated. 5. Structured handoff, transcript and timeline are persisted (UC-06).
Alternate / Exception Flow	If transcription confidence is low, the transcript is still stored but flagged for nurse review.
Post-conditions	A structured handoff record exists and is available for review.
Includes / Extends	Included by UC-01; includes SBAR generation, summary generation, timeline generation, and missing-information checking.

Use Case ID / Name	UC-03 / View AI Summary and Risk Indicator
Actor(s)	Nurse
Description	An incoming-shift nurse opens a patient profile to review the AI-generated summary, timeline, and deterioration-risk indicator.
Pre-conditions	A handoff has been processed and persisted (UC-02).
Main Flow	1. Nurse opens the patient profile. 2. System requests the AI summary. 3. System requests the timeline. 4. System requests the risk indicator. 5. Nurse reviews the combined view.
Alternate / Exception Flow	If risk-model output is unavailable, the summary and timeline are still shown with the risk section marked as pending.

Post-conditions	Nurse has reviewed the AI-generated handoff information for the shift.
Includes / Extends	Independent of UC-01; depends on UC-02 output.

Use Case ID / Name	UC-04 / Ask AI Assistant (RAG Query)
Actor(s)	Nurse
Description	A nurse asks a natural-language question (e.g. "What changed today?") and receives an answer grounded in that patient's stored records.
Pre-conditions	The patient has at least one stored handoff record.
Main Flow	1. Nurse types or speaks a question. 2. System retrieves semantically relevant patient records via vector search. 3. System supplies the retrieved context to the LLM. 4. Grounded answer is displayed to the nurse.
Alternate / Exception Flow	If no relevant record clears the retrieval threshold, the assistant returns a "no verified information available" response rather than an ungrounded answer.
Post-conditions	Nurse receives a patient-grounded answer.
Includes / Extends	Extends UC-03.

Use Case ID / Name	UC-05 / Edit Handoff Manually
Actor(s)	Nurse
Description	A nurse corrects or completes an AI-generated handoff summary before sending it to the next shift.
Pre-conditions	A structured handoff draft exists (UC-02).
Main Flow	1. Nurse opens "Edit Summary" on the Handoff screen. 2. Nurse edits SBAR fields or flagged missing fields. 3. Nurse saves changes. 4. Updated handoff is persisted and marked nurse-verified.
Alternate / Exception Flow	Nurse discards changes and keeps the AI-generated draft.
Post-conditions	A nurse-verified handoff record is stored.
Includes / Extends	Extends UC-02 and UC-03.

Use Case ID / Name	UC-06 / View Handoff Tasks and Send Handoff
Actor(s)	Nurse
Description	A nurse reviews the checklist of handoff-related tasks (e.g. medication administration, post-op vital check) for the current shift and sends the completed handoff to the incoming nurse.
Pre-conditions	The nurse is on an active shift with an assigned patient list.
Main Flow	1. Nurse opens the Handoff screen. 2. System displays open/done handoff tasks and the latest voice-log confidence. 3. Nurse marks tasks open/done. 4. Nurse taps "Send

	Handoff” to hand off to the next shift.
Alternate / Exception Flow	If no patient has been selected from the dashboard, the screen shows “No patient selected” and disables Send Handoff until one is chosen.
Post-conditions	The incoming nurse receives the completed handoff and task checklist.
Includes / Extends	Extends UC-01 and UC-05.

2.1.3 High-Level Design

Nursify follows a layered architecture in which the user-facing application is separated from the backend processing pipeline. The intended flow is: nurse input, mobile/desktop interface, FastAPI APIs, AI processing, persistent storage, and retrieval for subsequent users.

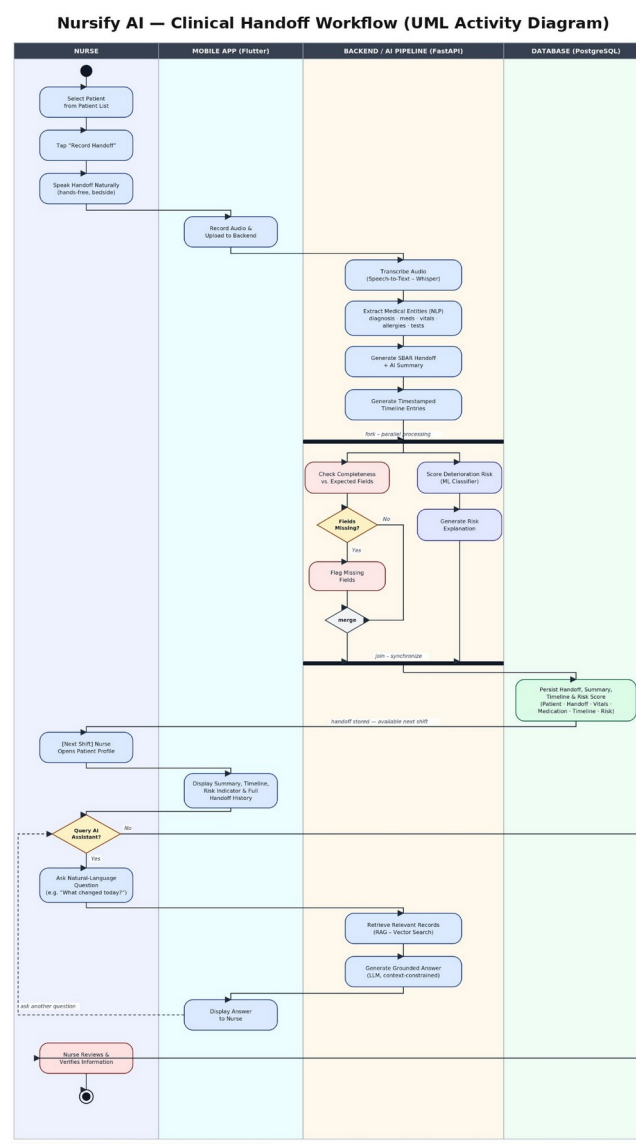


Figure 2.2: Nursify AI Clinical Handoff Workflow – UML Activity Diagram

The activity diagram shows the complete proposed workflow. The nurse selects a patient, starts a handoff, and speaks naturally. The application records and uploads audio. The backend performs speech-to-text, medical entity extraction, SBAR and summary generation, and timeline generation.

Completeness checking and deterioration-risk scoring are then designed to run in parallel before the resulting information is persisted. On a subsequent shift, the nurse can view the patient history and optionally ask a natural-language question through the RAG-assisted AI interface.

2.1.4 Sequence Diagram

The sequence diagram complements the activity diagram by showing the concrete request/response interactions between the nurse, the client application, the FastAPI backend, the speech-to-text (ASR) service, the NLP and handoff processor, the PostgreSQL database, and the AI assistant / risk model. It is organised into three phases: (1) authentication and patient selection, (2) recording and processing a voice handoff, and (3) viewing the AI summary, timeline, and risk indicator.

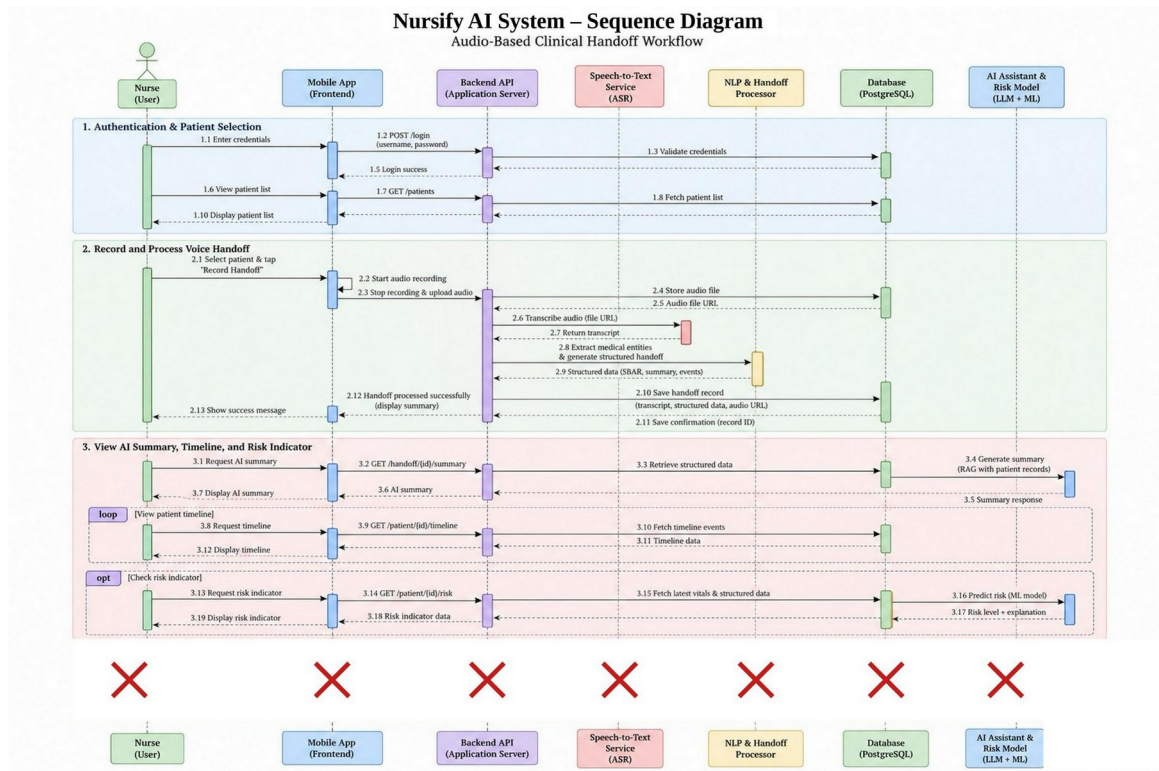


Figure 2.3: Nursify AI System – Sequence Diagram

This view makes explicit the API calls (for example POST /login, GET /patients, GET /handoff/{id}/summary) and the order in which the backend calls the ASR service, the NLP/handoff processor, and the database while a handoff is captured, structured, and later retrieved for review.

2.1.5 Collaboration Diagram

The collaboration (communication) diagram below presents the same handoff-capture interaction as the sequence diagram in Section 2.1.4, but from a structural viewpoint: objects are shown as linked nodes, with messages numbered in call order along the links that connect them, emphasising who talks to whom rather than strict time ordering. It makes explicit that the Mobile App never talks to the Auth Service, ASR, NLP Processor, or AI Assistant directly — every one of those collaborations is brokered by the Backend API, which also owns the create/update calls to the Patient and Handoff objects and the final persistence call to the database.

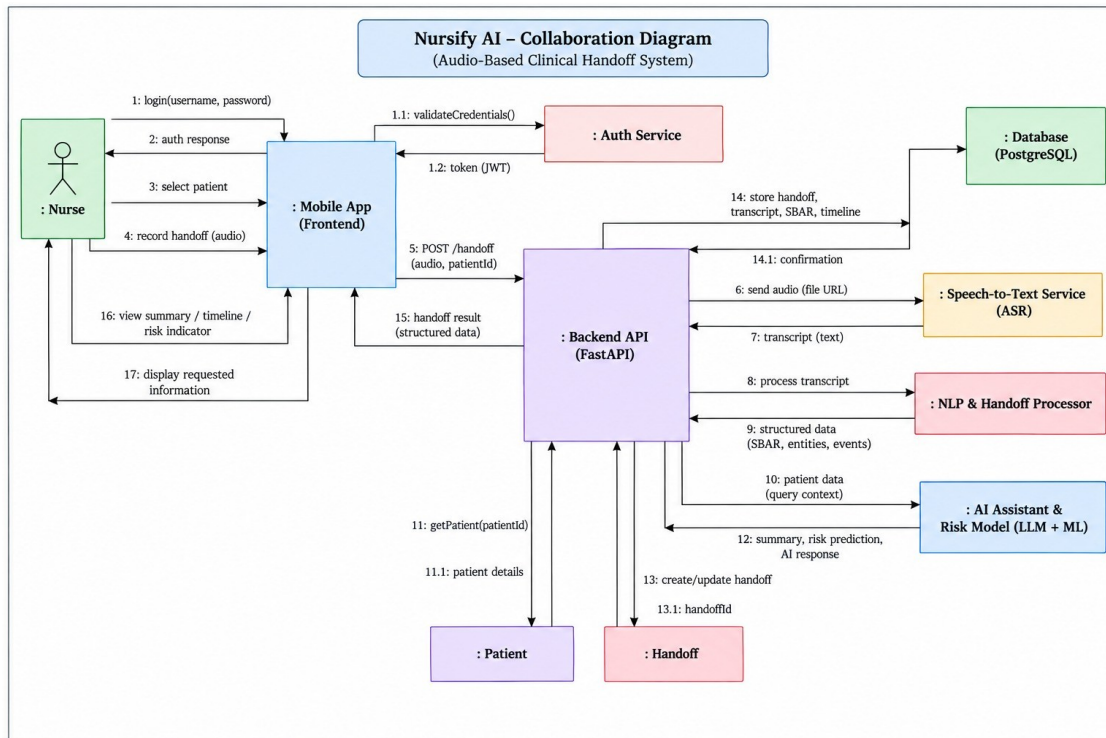


Figure 2.4: Nursify AI - Collaboration Diagram

2.1.6 Class Diagram

The class diagram defines the principal classes behind the planned data model and their relationships. A User (role NURSE, ADMIN, or DOCTOR) manages Patient records; each Patient has zero or more Handoff records. A Handoff comprises TimelineEvent entries and uses one AudioFile, which an external SpeechToTextService transcribes into a Transcript; the NLPProcessor generates an SBAR from that transcript, and a RiskModel uses the SBAR to predict a RiskLevel for the Handoff. An AIAssistant uses the Handoff/TimelineEvent history to answer nurse Query objects asked via the assistant.

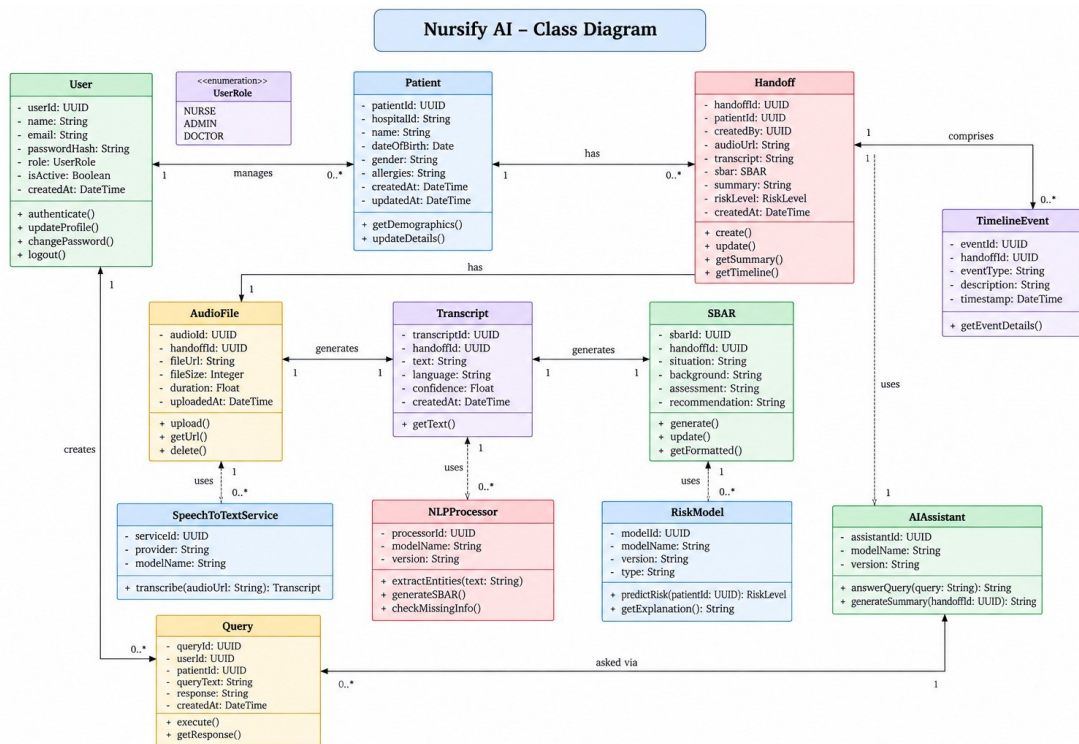


Figure 2.5: Nursify AI - Class Diagram

2.1.7 Entity-Relationship Diagram

The entity-relationship diagram defines the relational data model that underlies the planned PostgreSQL store. Users log handoffs for patients; a handoff generates timeline events, captures an audio recording, administers medications, and records vitals; vitals can trigger a risk assessment through threshold breaches, and a risk assessment also evaluates and concerns a specific patient.

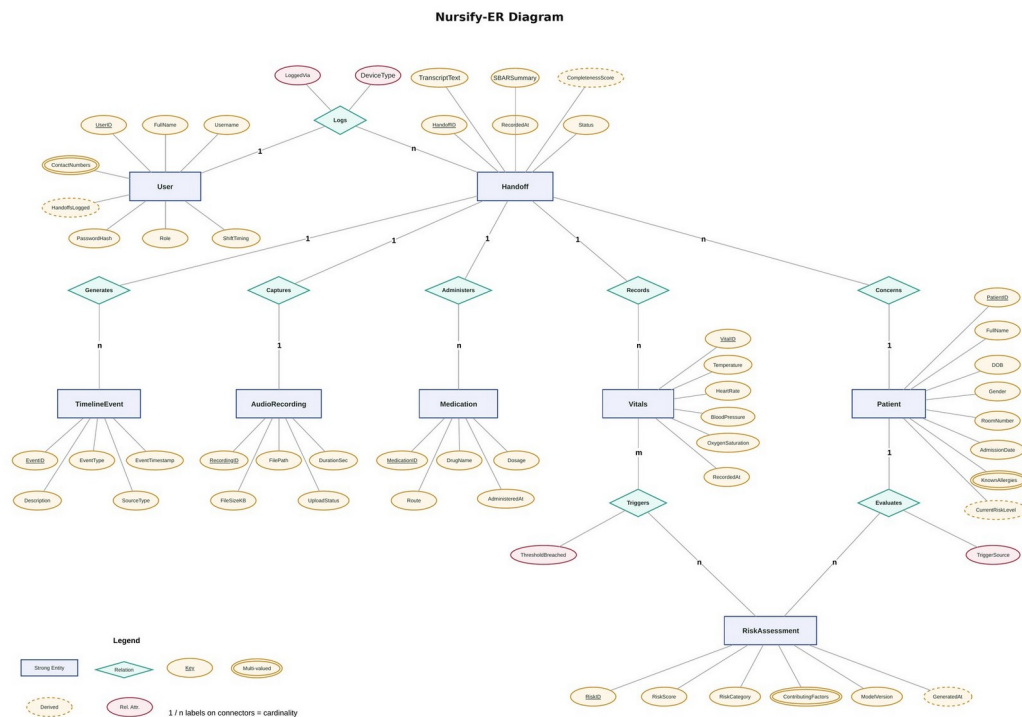


Figure 2.6: Nursify AI – Entity-Relationship Diagram

2.1.8 Technical Stack

The system uses mature components so that engineering effort can be concentrated on integration and correctness:

- Frontend: Flutter is the originally intended cross-platform application framework. The currently delivered prototype's multi-screen client (dashboard, task manager, handoff and patients tabs) runs as a lightweight webview-based desktop/web build served on the standard local development port used by this tooling, alongside a standalone speech-to-text web page used for the working demonstration.
- Backend: FastAPI provides the REST API layer and coordinates audio processing.
- Speech recognition: faster-whisper provides local speech-to-text inference. The current implementation runs the model locally on CPU using integer quantisation to reduce resource requirements.
- Data storage: PostgreSQL is the planned relational store for patients, handoffs, vitals, medications, timelines, and risk information.
- RAG layer: embeddings and vector search are planned for retrieving semantically relevant patient records before LLM answer generation.
- Machine learning: scikit-learn/XGBoost-style experiments are planned for deterioration-risk classification and explainability.

2.2 Data Flow Design

The Data Flow Diagrams divide the system into external actors, processes, data stores, and data flows. Level 0 treats Nursify AI as one system and shows its interaction with the nurse, administrator, and AI/ML services. Level 1 decomposes the system into handoff capture, handoff processing, data storage, AI assistant, and risk processing. Level 2 expands the handoff-processing stage into transcription, NLP extraction, SBAR/summary generation, timeline generation, risk prediction, and missing-information checking.

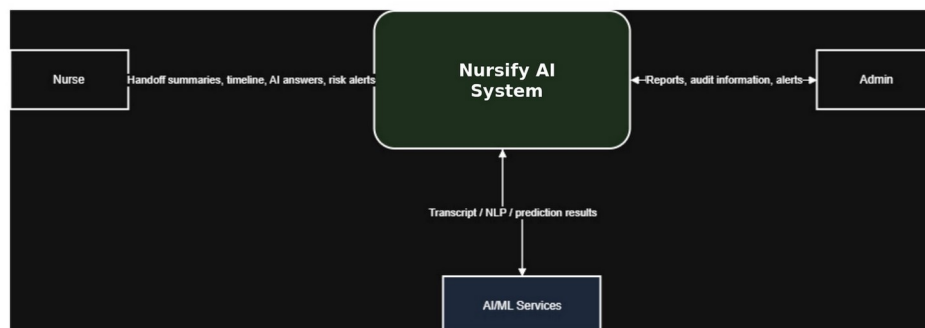


Figure 2.7: Nursify AI - Level 0 Context Data Flow Diagram

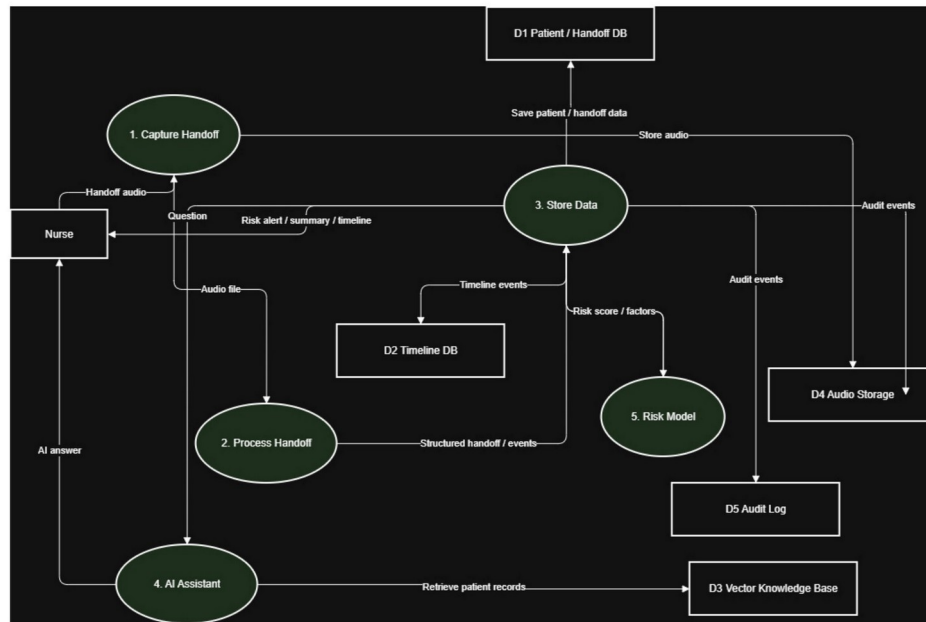


Figure 2.8: Nursify AI - Level 1 Data Flow Diagram

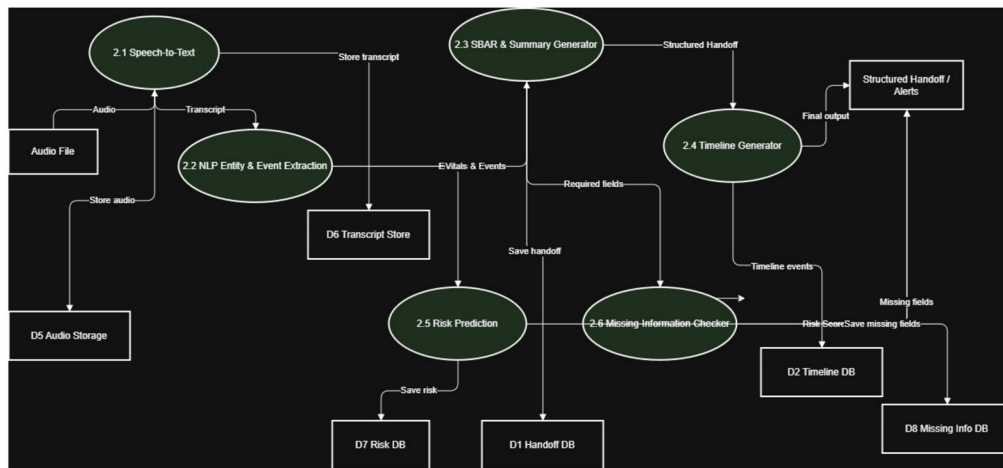


Figure 2.9: Nursify AI - Level 2 Handoff Processing Data Flow Diagram

2.2.1 Handoff Processing Pipeline

The detailed processing pipeline is designed as:

- Audio input: a nurse records a spoken handoff.
- Speech-to-text: the audio is passed to faster-whisper and converted into a transcript.

- Medical entity extraction: an NLP/LLM stage identifies clinically meaningful information.
- SBAR and summary generation: extracted information is organized into Situation, Background, Assessment, and Recommendation fields and a concise summary.
- Timeline generation: clinically relevant events are converted into timestamped timeline entries.
- Completeness checking: expected fields are compared with the extracted information and missing items can be flagged.
- Risk prediction: an experimental classifier produces a deterioration-risk score and an explanation of contributing factors.
- Persistence: structured handoff information, summaries, timeline events, and risk information are stored for later retrieval.

2.3 Software Requirement Specification (IEEE Format)

2.3.1 Introduction

Purpose. This Software Requirements Specification (SRS) describes the functional and non-functional requirements of Nursify AI, a voice-driven clinical handoff and decision-support system. It is intended for use by the development team, the course faculty evaluating the mid-semester submission, and any future maintainers of the system.

Scope. Nursify AI will allow a nurse to record a spoken bedside handoff and receive a structured, AI-generated SBAR summary, patient timeline, completeness check, and experimental deterioration-risk indicator, together with a retrieval-augmented AI assistant for patient-specific questions. It will maintain a shift-level task manager for handoff-related nursing tasks. The system explicitly excludes: autonomous diagnosis, prescription of medication or dosages, and any real-time integration with emergency dispatch or life-support systems — all AI outputs are decision-support only and require nurse verification.

Definitions, Acronyms and Abbreviations:

- RAG — Retrieval-Augmented Generation.
- ASR — Automatic Speech Recognition.
- SBAR — Situation, Background, Assessment, Recommendation.
- LLM — Large Language Model.
- Grounded answer — a response constructed only from retrieved, verified patient records, with the LLM used for language generation rather than as a source of facts.

References:

- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications.
- Project artefacts: Use-Case Diagram and Templates, Activity, Sequence, Collaboration and Class Diagrams, Entity-Relationship Diagram, and DFD Levels 0–2 (Section 2.1–2.2 of this report).

2.3.2 Overall Description

Product Perspective. Nursify AI is a new, self-contained clinical documentation system. It integrates with local/hosted AI services — a speech-to-text engine (faster-whisper), an NLP/LLM handoff processor, and an experimental risk model — and maintains its own relational store (PostgreSQL, planned) and audio/transcript storage rather than depending on any existing hospital information system.

Product Functions.

- Capture a spoken handoff and convert it to a transcript.
- Structure the transcript into SBAR fields, a summary, and a timestamped timeline.
- Flag missing expected fields and provide an experimental deterioration-risk score.
- Allow a nurse to browse patients, review handoff history, and edit a handoff manually.
- Answer natural-language, patient-grounded questions through a RAG-based AI assistant.
- Track AI-prioritised shift tasks (e.g. medication administration, vitals checks) alongside the handoff.

User Classes and Characteristics.

- Nurse (primary user) — clinical staff with no software-engineering background; interacts through the mobile/desktop app and voice input.
- Admin / Charge Nurse (future role) — oversees patient assignment and audit information; not yet implemented in the current prototype.

Operating Environment. The system runs as a cross-platform client (currently built and demonstrated as a lightweight desktop/web application) backed by a Python FastAPI service hosting the speech-to-text, NLP, and (planned) RAG pipeline, with PostgreSQL for persistent storage. It is expected to run on standard hospital workstation or tablet hardware; no specialised hardware is required beyond a microphone.

Design and Implementation Constraints.

- Speech-to-text runs locally (faster-whisper) so that no recorded patient audio needs to leave the local environment for basic transcription.
- The AI assistant must only answer from retrieved, verified patient records; ungrounded answers are explicitly out of scope.
- All AI-generated content (summary, risk indicator) must remain editable and nurse-verifiable before being treated as the shift-of-record handoff.

Assumptions and Dependencies.

- A working microphone and network connectivity to the backend are available on the nurse's device.
- The initial clinical knowledge and terminology used for entity extraction are drawn from credible, de-identified reference material during development.

2.3.3 Specific Requirements

Functional Requirements

ID	Requirement
FR-1	The system shall allow an authenticated nurse to select a patient and record a spoken handoff.
FR-2	The system shall upload the recorded audio and transcribe it into text using a local speech-to-text engine.
FR-3	The system shall extract clinically meaningful entities (vitals, medications, diagnoses, allergies, tests) from the transcript.
FR-4	The system shall generate a structured SBAR handoff and a concise summary from the extracted entities.
FR-5	The system shall generate timestamped timeline events for clinically relevant occurrences during the shift.
FR-6	The system shall compare extracted information against expected fields and flag missing items for nurse attention.
FR-7	The system shall produce an experimental deterioration-risk score with a contributing-factor explanation.
FR-8	The system shall allow a nurse to view the AI summary, timeline, and risk indicator for a selected patient.
FR-9	The system shall allow a nurse to manually edit an AI-generated handoff before it is sent to

ID	Requirement
	the next shift.
FR-10	The system shall answer a nurse's natural-language question using retrieval-augmented generation grounded in that patient's stored records.
FR-11	The system shall maintain a shift task list (e.g. medication administration, vitals checks) and let the nurse mark tasks open or done.

Non-Functional Requirements

- **Safety:** the system shall never present an AI-generated summary or risk score as a substitute for clinical judgement; every AI output remains nurse-editable before being finalised (see UC-05).
- **Reliability:** core recording and transcription shall remain usable even when downstream AI stages (SBAR generation, risk scoring, RAG assistant) are degraded or unavailable.
- **Usability:** the recording flow shall require no more than patient selection and a single tap to start capturing a handoff.
- **Maintainability:** the layered architecture (client / FastAPI / AI services / PostgreSQL) shall allow each AI stage to be replaced or upgraded independently.
- **Performance:** transcription and structuring of a typical bedside handoff should complete within a few seconds to a small number of seconds under normal load.
- **Traceability:** every handoff, edit, and AI assistant query shall be timestamped and stored to support later audit and evaluation.

2.4 User Stories and Story Cards

The following user stories translate the use cases in Section 2.1.2 into the standard agile “As a / I want to / so that” format, each with acceptance criteria, priority, and a rough size estimate in story points.

US-1: Record a handoff hands-free

As a nurse, I want to record my shift handoff by speaking naturally, so that I don't have to stop and type while caring for patients.

Acceptance Criteria: Given a selected patient, tapping “Record Handoff” captures audio and uploads it for processing without requiring any typed input.

Priority: Highest | Estimate: 8 story points

US-2: Review an AI-generated summary and timeline

As an incoming-shift nurse, I want to see a structured summary and timeline for my assigned patient, so that I can quickly understand what happened on the previous shift.

Acceptance Criteria: Given a processed handoff, opening the patient profile shows the SBAR summary and a timestamped timeline within the same screen.

Priority: High | Estimate: 5 story points

US-3: Ask the AI assistant a question

As a nurse, I want to ask a plain-language question about my patient, so that I can find specific information without scrolling through the whole handoff history.

Acceptance Criteria: Given a patient with at least one stored handoff, asking a question returns an answer grounded in that patient's records, or a clear “no verified information” message.

Priority: High | Estimate: 8 story points

US-4: See flagged missing information

As a nurse, I want the system to flag information I forgot to mention, so that I can add it before ending my shift.

Acceptance Criteria: Given an expected field (e.g. allergies) is absent from the transcript, the handoff draft shows it as a flagged missing field.

Priority: Medium | Estimate: 5 story points

US-5: View the deterioration-risk indicator

As a nurse, I want to see a risk indicator alongside the handoff, so that I can prioritise attention for patients who may be deteriorating.

Acceptance Criteria: Given vitals and structured handoff data exist for a patient, the risk indicator displays a risk level with a short explanation of contributing factors.

Priority: Medium | Estimate: 8 story points

US-6: Correct an AI-generated handoff

As a nurse, I want to edit the AI-generated SBAR summary before sending it on, so that I remain responsible for the accuracy of the handoff.

Acceptance Criteria: Given a handoff draft, the nurse can edit any SBAR field and save the corrected version, which is then marked nurse-verified.

Priority: High | Estimate: 3 story points

US-7: Work an AI-prioritised task list

As a nurse, I want my overdue and high-risk tasks surfaced at the top of my task list, so that nothing urgent is missed during a busy shift.

Acceptance Criteria: Given open tasks with due times, overdue and high-risk tasks (e.g. medication administration, post-op vital checks) are shown first and clearly marked.

Priority: High | Estimate: 5 story points

Chapter 3

Implementation and Evaluation

3.1 Implementation Status

The project is being delivered incrementally. The current repository and build combine a working speech-to-text demonstration with a multi-screen client shell (Dashboard, Handoff, Tasks, and Patients tabs) rather than the fully wired end-to-end clinical application. The implemented work includes the FastAPI application setup, local faster-whisper integration, the transcription endpoint, a speech-to-text frontend page, the navigable client shell described in Section 3.1.4, and end-to-end verification from recording through transcription and display.

The broader PostgreSQL-backed clinical data model, RAG assistant, completeness checker, and risk model remain part of the staged development roadmap. They are therefore described in this report as planned or architectural components rather than as completed production functionality.

3.1.1 FastAPI Backend and CORS

FastAPI acts as the server-side entry point. The backend exposes an HTTP endpoint that accepts uploaded audio, passes the file to the transcription model, and returns a formatted transcript to the frontend.

For browser-based development, Cross-Origin Resource Sharing (CORS) is configured using FastAPI's CORSMiddleware. CORS allows the backend to explicitly control which frontend origins, methods, headers, and credentials may be used for cross-origin requests. A permissive configuration can be used during local development, while a production deployment should restrict the allowed origins to trusted clients.

A simplified configuration is:

```
from fastapi import FastAPI from fastapi.middleware.cors import
CORSMiddleware app = FastAPI() app.add_middleware(      CORSMiddleware,
allow_origins=["*"],      allow_credentials=True,      allow_methods=["*"],
allow_headers=["*"], )
```

The important implementation idea is that CORS is a communication/security mechanism, not an AI component. It allows a browser-based client to call the FastAPI service when the request origin satisfies the configured policy.

3.1.2 Speech-to-Text Implementation

The speech-to-text backend uses the faster-whisper implementation of Whisper for local inference. The model is deliberately run locally so that the prototype does not require a hosted transcription API key or a per-request transcription service. Once the model is available locally, transcription can also be performed without sending the recorded audio to an external transcription provider.

The end-to-end flow is: Record Audio → Upload → FastAPI → faster-whisper → Transcript → Display

The frontend exposes clear recording states so that the user can distinguish between recording and transcription. The current speech-to-text page captures microphone audio, uploads the resulting recording, and displays the returned transcript.

3.1.3 RAG and Vector Search Design

The proposed AI assistant uses Retrieval-Augmented Generation (RAG). When a nurse asks a question such as “What changed today?”, the system first retrieves relevant patient records through vector search. The retrieved information is then supplied to an LLM as context, allowing the generated answer to remain grounded in the patient's available records.

Text or record chunks can be transformed into numerical embeddings. Vector search compares the embedding of the nurse's question with stored embeddings and returns semantically relevant records. This differs from a simple keyword search because related concepts such as “oxygen level” and “SpO2” can be represented as semantically similar information.

3.1.4 Prototype User Interface (Current Build)

Beyond the standalone speech-to-text page, a navigable client shell has been built covering the four main screens implied by the use-case and activity diagrams: Dashboard, Handoff, Tasks, and Patients. The current build runs as a local desktop/web application during development and uses placeholder/demo data (an evening ICU shift for “Nurse Priya”) while the backend integration for the full pipeline is completed.

Dashboard. The dashboard shows shift context (shift type and ward), a patient counter, an open-task counter, and shift-completion progress, together with a patient selector and a “Voice Log Center” microphone control that starts a handoff recording once a patient has been selected.

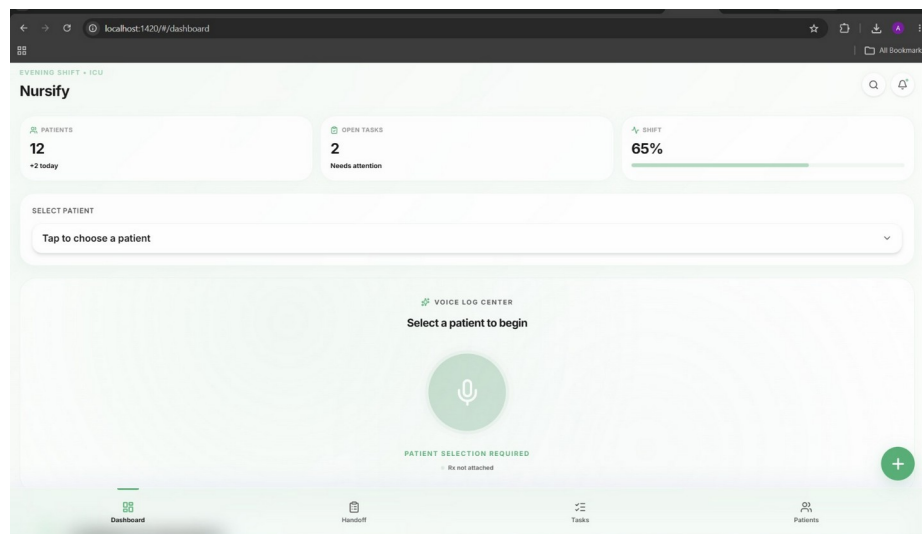


Figure 3.1: Prototype Dashboard screen — shift summary, patient selector, and Voice Log Center

Task Manager. The task manager presents an AI-prioritised list of nursing tasks for the shift, splitting out urgent/overdue items — for example an overdue medication administration and a post-op vital check, both flagged “OVERDUE - HIGH RISK” — from completed tasks such as a respiratory assessment, mirroring the completeness-checking and risk-prediction concepts described in Sections 2.1.2 and 2.2.1.

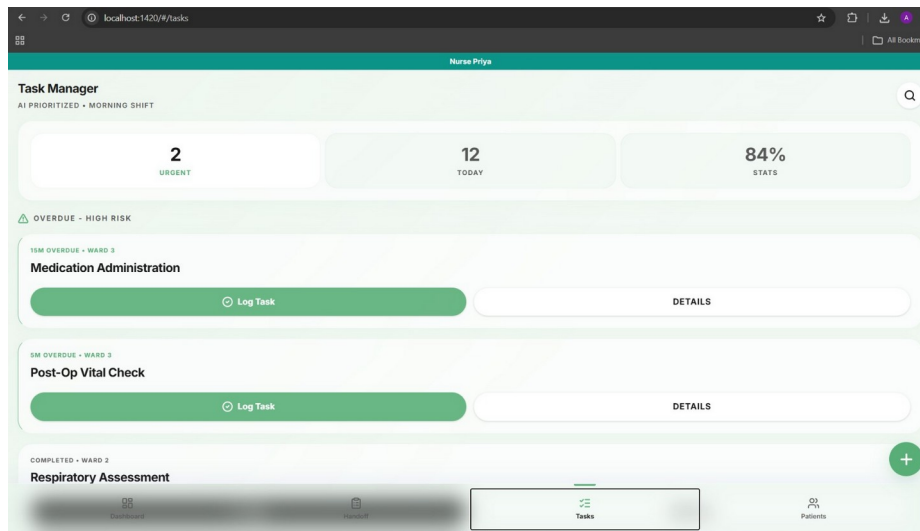


Figure 3.2: Prototype Task Manager screen — AI-prioritised, overdue/high-risk tasks

Handoff Summary. The handoff screen shows the current patient context, any pending incoming handoff, the handoff task checklist (open/done), and the latest voice-log confidence from the most recent recording, with “Edit Summary” and “Send Handoff” actions corresponding to UC-05 and UC-06 respectively.

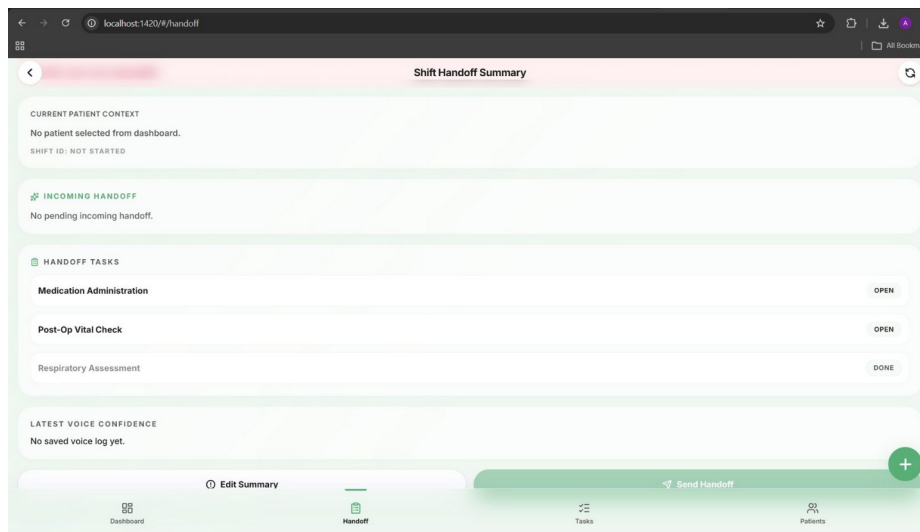


Figure 3.3: Prototype Handoff Summary screen — task checklist, voice confidence, edit and send actions

These three screens demonstrate the client-side navigation shell and UI states (including empty/pending states such as “No patient selected” and “No pending incoming handoff”) that the backend pipeline described in Section 2.2.1 will populate as each AI stage is completed.

3.2 Testing Methodology

Testing of the current delivered prototype focuses on the speech-to-text path and the client navigation shell. The end-to-end test verifies the complete chain from microphone recording to transcription and display, and manual walkthroughs verify that the Dashboard, Handoff, and Tasks tabs render their expected empty and populated states. Backend testing also checks that the FastAPI service can receive the uploaded audio and return a usable response.

The planned evaluation framework expands this into multiple measurable dimensions:

- Handoff Documentation Time: time from recording start to a fully structured and stored handoff.
- Entity extraction accuracy: precision, recall, and F1 for fields such as diagnosis, medication, vitals, allergies, and tests.
- Completeness detection: precision and recall for intentionally omitted handoff fields.
- RAG groundedness: proportion of assistant answers fully supported by retrieved patient records.
- Risk model performance: accuracy, precision, recall, F1, and ROC-AUC, with particular attention to sensitivity and false negatives.
- Usability: nurse feedback on the summary, timeline, task manager, and AI-assistant usefulness.

3.3 Current Results

The current prototype demonstrates two things: (1) the core technical feasibility of the first AI stage — a recorded voice handoff can be sent to the FastAPI backend, processed locally with faster-whisper, and returned as text for display — and (2) a working, navigable client shell (Dashboard, Handoff, Tasks, Patients) that establishes the screens and states the remaining AI stages will populate. Together these validate both the audio-to-transcript communication path and the intended user-facing structure required for later clinical structuring stages.

Current implementation boundary: the report intentionally distinguishes implemented functionality from proposed architecture. The current delivered build centres on the standalone speech-to-text demonstration and the client navigation shell shown in Section 3.1.4; the full PostgreSQL clinical workflow, RAG assistant, and deterioration-risk pipeline are subsequent development stages.

Chapter 4

Conclusion

4.1 Summary of Work

Nursify AI establishes an engineering foundation for voice-driven clinical handoff documentation. The project models the workflow using UML use case, activity, sequence, collaboration, and class diagrams together with data-flow diagrams, an entity-relationship diagram, an IEEE-format SRS, and a set of user stories, and defines a staged architecture connecting a user interface, FastAPI backend, speech recognition, clinical AI processing, relational storage, and retrieval-assisted question answering.

The first working implementation focuses on speech-to-text together with a navigable client shell. The team has set up the FastAPI service, integrated faster-whisper for local transcription, implemented the transcription endpoint and a frontend recording page, built Dashboard/Handoff/Tasks/Patients screens for the client, and verified the end-to-end recording-to-transcript flow.

4.2 Key Findings

- Local faster-whisper transcription provides a practical first AI component without requiring a hosted transcription API.
- Separating the frontend from the FastAPI backend gives the project a clean boundary for later AI and database integration.
- The use case, DFD, activity, sequence, collaboration, class, and ER diagrams together make the planned clinical pipeline and its underlying data model explicit, providing a roadmap for incremental implementation.
- Writing the requirements as an IEEE-format SRS and as user stories surfaced concrete, testable acceptance criteria (e.g. for missing-field flagging and the risk indicator) that the narrative description alone did not make explicit.
- RAG can provide a safer patient-specific question-answering architecture by retrieving relevant records before asking an LLM to generate an answer.
- Human verification remains essential because transcription, extraction, summarization, retrieval, and risk models can all produce errors.

4.3 Future Work and Recommendations

The next development stages are:

- Integrate the patient and handoff data model with PostgreSQL.
- Add medical entity extraction and structured clinical fields.
- Generate SBAR handoffs, concise summaries, and timestamped timelines.
- Implement the missing-information checker and review/correction workflow.
- Build the RAG assistant with embeddings and vector retrieval over patient records.
- Develop and evaluate the experimental deterioration-risk classifier with explainability.
- Wire the Dashboard, Handoff, and Tasks screens shown in Section 3.1.4 to the live backend pipeline instead of placeholder data.
- Add stronger authentication, access control, audit logging, automated tests, CI/CD, and secure deployment before any real clinical use.

4.4 Final Remarks

Nursify AI is designed to reduce documentation friction while improving the accessibility and continuity of clinical handoff information. The current speech-to-text prototype and client shell provide a concrete first increment, while the architecture and design artifacts — including the newly added collaboration diagram, class diagram, SRS, and user stories — establish a path toward structured handoffs, searchable timelines, grounded AI assistance, and explainable decision support. Any future clinical deployment must use appropriate privacy safeguards, synthetic or de-identified development data, human verification, and rigorous evaluation before use with real patient information.

Bibliography

- [1] A. Radford et al., "Robust Speech Recognition via Large-Scale Weak Supervision," OpenAI, 2022.
- [2] FastAPI Documentation, "FastAPI – Modern, fast web framework for building APIs with Python."
- [3] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems, 2020.
- [4] PostgreSQL Global Development Group, "PostgreSQL Documentation."
- [5] Flutter Documentation, "Build apps for any screen."
- [6] IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications.
- [7] Nursify AI Team, "Nursify AI – Intelligent Audio-Based Clinical Handoff System," Project Proposal, Thapar Institute of Engineering and Technology, 2026.